SURGITRACK iLog – MASTER FUNCTIONAL DEVELOPMENT & SYSTEM ACTIVATION PROMPT

Advanced Backend Engineering, Database Architecture, Authentication, Surgical Community & Portfolio Management

1. ROLE AND DEVELOPMENT OBJECTIVE

Act as a world-class:

- Senior Full-Stack Software Architect.
- Backend and Database Engineer.
- Firebase / Cloud Infrastructure Specialist.
- Authentication and Application Security Engineer.
- Real-Time Social Platform Developer.
- Medical Portfolio and Clinical Data Systems Engineer.
- Professional UI/UX Engineer.

You are working directly on the existing SurgiTrack iLog application.

Your mission is to inspect, repair, and activate the existing features described below so they work as real application functionality, not merely visual interface elements.

The existing application design is already established.

Preserve the current UI, layout, colors, typography, icons, page structure, and visual identity. Do not redesign existing screens.

Only modify or add the minimum interface elements necessary to make the requested functionality work.

CORE EXECUTION PRINCIPLE

Every button, icon, action, and feature described in this specification must be connected to a real backend operation and persistent database functionality.

Do not implement fake interactions, temporary local-only changes, mock data, or buttons that merely display success messages.

All changes must persist after refreshing the application, signing out, and signing in again.

Before making changes, inspect the actual project structure, existing code, database, authentication provider, storage provider, and deployed infrastructure.

Reuse the existing architecture wherever possible. Do not rebuild the application or replace working systems unnecessarily.

2. FEATURE ONE – FULLY ACTIVATE FILE AND RECORD DELETION

Current problem

In the Logbook Entries section, every uploaded or added file has a delete icon next to it.

However, the delete icon is currently not clickable or does not execute the deletion operation.

Consequently, users cannot remove files or materials they have added to their portfolios.

The same problem affects the ability to remove clinical cases and published community posts.

Required implementation

Activate every existing delete icon associated with:

- Uploaded files.
- Documents attached to Logbook Entries.
- Images and other uploaded materials.
- Clinical case attachments.
- Other user-added portfolio materials.
- Clinical cases created by the user.
- Community posts created by the user, where deletion is permitted.

A. File deletion workflow

When the user clicks a delete icon:

1. Identify the exact file or attachment.

2. Confirm that the current authenticated user owns the resource or has authorized administrative permission.

3. Display a professional confirmation dialog before permanent deletion.

4. When confirmed, delete the physical file from the permanent storage provider.

5. Delete the corresponding database record and all references to that file.

6. Update the interface immediately.

7. Ensure the deleted file no longer appears in Logbook Entries or any other relevant portfolio section.

8. Ensure the deleted file does not reappear after refreshing or logging in again.

B. Permanent deletion requirements

Deletion must affect both:

- The persistent storage object.
- The database record referencing the object.

Do not merely hide the file from the interface.

Do not delete the database record while leaving an inaccessible storage object behind.

Do not delete files belonging to another user.

If a deletion operation fails, retain the existing record and display a clear error message rather than falsely reporting success.

Use safe deletion handling and prevent orphaned files or broken references.

C. Clinical case deletion

Allow users to delete clinical cases they personally created.

When deleting a clinical case:

- Remove it from the user's portfolio.
- Remove its associated attachments where appropriate.
- Remove its community publication if the user explicitly chooses to delete the entire case.
- Update the Published Posts list.
- Update relevant community feeds.
- Prevent deleted cases from appearing in dashboards or portfolio exports.

Where a case has both a private portfolio record and a community publication, handle both records consistently.

Provide a clear confirmation step to prevent accidental deletion.

D. Community post deletion

Allow users to delete their own published community posts.

The deletion must remove the publication from:

- The user's Published Posts list.
- The Surgical Community feed.
- Followers' relevant feeds.
- Any associated public post view.

Maintain proper authorization and prevent users from deleting other users' posts.

3. FEATURE TWO – STRICT EMAIL VERIFICATION DURING REGISTRATION

Objective

Implement mandatory email verification for all newly registered accounts.

The platform must verify that the email address supplied during registration is real and accessible to the user.

The verification process must use a real clickable email link, not OTP or manual verification codes.

Required registration workflow

1. User opens the registration page.

2. User enters the required registration details.

3. User submits the registration form.

4. The system creates the account in an unverified state.

5. The system generates a secure verification link.

6. The system sends the verification email to the exact email address entered during registration.

7. The user opens the email and clicks the verification link.

8. The backend validates the verification action.

9. The account becomes verified only after successful server-side validation.

10. The user can then access the protected application features.

Mandatory access restrictions

Unverified users must not be allowed to access protected application functionality.

Restrict access to:

- Surgical Portfolio.
- Logbook Entries.
- Clinical Cases.
- Surgical Skills.
- Dashboard.
- Surgical Community.
- Publishing.
- Following and social interactions.
- Notifications.
- PDF Export.
- Other protected application areas.

Show a clear verification-required screen with:

- A message explaining that email verification is required.
- The destination email address.
- A Resend Verification Link action.
- A Refresh Verification Status action.
- A Sign Out action.

Existing accounts

Inspect the existing user database before applying mandatory verification.

Do not accidentally lock out legitimate existing users.

Preserve the current administrator account and existing account data.

Apply a safe migration strategy for existing users according to their actual verification status.

Do not mark an unverified email as verified without a legitimate verification process.

4. CRITICAL SECURITY REQUIREMENT – REAL EMAIL VERIFICATION LINK ONLY

Mandatory Email Verification – No OTP, No Verification Codes

Implement a real, server-side email verification system for every newly registered SurgiTrack user.

This requirement is mandatory and must not be simulated.

A. Registration workflow

When a new user registers:

1. Accept the email address submitted in the registration form.

2. Create the account in an unverified state.

3. Generate a cryptographically secure, unique, single-use verification token.

4. Generate a verification URL associated exclusively with that account and its registered email address.

5. Send an actual verification email to the exact email address entered during registration.

6. The email must contain a clickable verification link.

7. The user must open that link to verify the account.

8. The backend must validate the token and confirm email ownership.

9. Only after successful verification should the account become verified and gain access to protected application features.

B. STRICT 30-SECOND LINK EXPIRATION

The verification link must expire exactly 30 seconds after it is generated.

Requirements:

- Store the token creation timestamp on the server.
- Enforce expiration using server-side time.
- If the user clicks the link within 30 seconds, validate it and verify the account.
- If the user clicks after 30 seconds, reject the expired link.
- An expired link must never verify an account.
- The expiration must not depend on the user's device clock.
- The verification token must be single-use.
- Once successfully used, it must immediately become invalid.
- Prevent replay attacks and token reuse.

Use a secure token implementation and store only a cryptographic hash of the token where appropriate.

The 30-second validity period must be enforced by the backend. Do not replace it with a longer period or silently extend it.

C. NO OTP OR VERIFICATION CODE IN THE USER INTERFACE

This is an absolute requirement.

Do not display:

- OTP input fields.
- Verification code input fields.
- Six-digit verification codes.
- Manual verification code forms.
- Any code that the user must copy from their email and enter into the website.

The entire verification process must happen through the clickable email link.

The registration interface must only display a message such as:

"Your account has been created. A verification link has been sent to your registered email address. Please open the link within 30 seconds to activate your account."

The user must not be asked to enter any verification code.

D. STRICT EMAIL DESTINATION VALIDATION

The verification email must be sent to the exact email address used during registration.

For example:

If the registration email is user@example.com, the verification email must be delivered to user@example.com.

Do not send verification links to:

- An administrator's email.
- A predefined email address.
- A different email stored in a static configuration.
- An unrelated account.
- A default testing email.

Associate every verification token with the correct user ID and registered email address.

Before accepting verification, confirm that:

- The token exists.
- The token belongs to the correct account.
- The token is associated with the account's current registered email.
- The token has not expired.
- The token has not already been used.
- The account has not already been verified through another valid process.

If the registered email address changes, invalidate outstanding verification tokens associated with the previous email address and require verification of the new address.

E. SECURE BACKEND VERIFICATION

Do not implement verification using frontend-only state.

The backend must independently validate the token and expiration.

On successful verification:

1. Mark the registered email as verified in the authentication provider.

2. Update the corresponding user record consistently.

3. Invalidate the token.

4. Prevent further use of that token.

5. Redirect the user to the appropriate SurgiTrack login or application page.

6. Allow access only after the backend confirms the verified account status.

Do not trust query parameters or frontend flags that claim an account is verified.

F. EXPIRED LINK HANDLING

If the user opens the link after 30 seconds, display a clear message:

"This verification link has expired. Please request a new verification link."

Provide a Resend Verification Link action.

When the user requests a new link:

- Generate a new secure token.
- Invalidate previous unused tokens for that account.
- Send the new link to the same registered email address.
- Start a new 30-second expiration period.

- Apply rate limiting to prevent abuse.

Do not silently extend the validity of an expired token.

G. EMAIL DELIVERY – REAL INTEGRATION REQUIRED

Inspect the existing authentication and email infrastructure.

Use the existing supported email delivery service if available.

If an email provider or SMTP configuration is missing, identify the exact required configuration.

Do not simulate email delivery.

Do not report "Email sent successfully" unless the email provider has accepted the delivery request successfully.

Log delivery failures safely without exposing verification tokens or sensitive information.

Use HTTPS for verification links.

Ensure that the email delivery implementation does not accidentally expose tokens in application logs, analytics, or error reports.

H. STRICT ACCESS CONTROL

Until the email verification process succeeds, the user must not access protected SurgiTrack features.

Enforce this on the backend, including:

- Surgical Portfolio.
- Logbook Entries.
- Clinical Cases.
- Surgical Skills.
- Dashboard.
- Surgical Community.
- Publishing.
- Following.
- Notifications.
- PDF Export.

Frontend route protection alone is insufficient.

Every protected backend API, server action, and database operation must independently enforce the verified-account requirement.

I. REAL-WORLD ACCEPTANCE TEST

Do not declare this feature complete without performing a real email delivery test.

Use a real test email inbox.

Perform the following:

1. Register a new test account using a real accessible email address.

2. Inspect the actual inbox and confirm that a verification email arrives.

3. Confirm that the recipient is exactly the registered email address.

4. Open the email and click its verification link within 30 seconds.

5. Confirm that the account becomes verified.

6. Confirm that protected features become accessible.

7. Register another test account.

8. Allow its verification link to expire beyond 30 seconds.

9. Click the expired link.

10. Confirm that verification is rejected.

11. Request a new link.

12. Confirm that the new link is delivered to the same registered email address.

13. Confirm that the new link works within its validity period.

14. Confirm that previously used links cannot be reused.

Test the complete workflow using the deployed application and actual email delivery infrastructure, not just a local mock environment.

NON-NEGOTIABLE: SurgiTrack must use real email-link verification, with a strict 30-second server-enforced expiration. No OTP, no manual verification codes, and no fake email delivery.

5. FEATURE THREE – FUNCTIONAL SAVE AND PUBLISH OPTIONS FOR CLINICAL CASES

Current problem

When creating a new clinical case, the interface displays two options:

■ Save & Publish to Community

■ Save to Portfolio Only

However, these options currently do not perform their intended operations correctly.

Selecting Save & Publish may save the case privately without actually publishing it to the community.

This must be fixed.

Required functionality

The two options must perform different and clearly defined backend operations.

OPTION A – Save to Portfolio Only

When selected:

1. Save the clinical case to the authenticated user's private portfolio.

2. Save all associated clinical information and attachments.

3. Set the publication state to private.

4. Do not publish the case to Surgical Community.

5. Do not display the case in other users' community feeds.

6. Do not notify followers about the case.

7. Display the case in the owner's portfolio and relevant private records.

8. Allow the owner to edit or delete it according to existing permissions.

The case must remain private unless the owner explicitly publishes it later.

OPTION B – Save & Publish to Community

When selected:

1. Validate the clinical case information.

2. Save the case to the owner's portfolio.

3. Create or activate the associated community publication.

4. Set the publication state to published.

5. Make the case visible in Surgical Community according to the application's visibility and access rules.

6. Display the case in the owner's Published Posts list.

7. Make the case available to eligible followers.

8. Allow authorized users to open and interact with the publication.

9. Trigger notifications for eligible followers.

10. Ensure that the portfolio record and community publication are correctly linked.

A published clinical case must exist in both places:

- The owner's personal portfolio.
- The Surgical Community publication system.

Do not create duplicate clinical case records simply because the user selected Save & Publish.

Use a reliable relationship between the portfolio case and its community publication.

Publishing reliability

Implement proper backend transaction handling or an equivalent reliable workflow.

If saving succeeds but publishing fails, the application must not falsely tell the user that the case was published.

Show the actual status and provide a safe retry option.

Prevent duplicate publications when a user clicks the publish action multiple times.

6. FEATURE FOUR – FULLY FUNCTIONAL SURGICAL COMMUNITY

Objective

Activate Surgical Community as a real professional networking and knowledge-sharing environment for surgeons and surgical trainees.

The existing Surgical Community icon and interface have already been designed.

Do not redesign them.

Your responsibility is to connect the existing interface to real users, follow relationships, publications, notifications, and interactions.

A. Discoverable user profiles

Whenever a new verified user registers, the system must create a discoverable professional profile.

Within Surgical Community, display the user's:

- Profile avatar or account icon.
- Display name.
- Professional title or qualification, when provided.
- Specialty, when provided.
- Relevant public professional profile information.

The profile should appear in a discoverable community members section.

Important privacy requirement

Only display information the user is permitted to share publicly.

Never expose:

- Email addresses by default.
- Private contact information.
- Passwords or authentication tokens.
- Patient-identifying clinical information.
- Private portfolio records.
- Other confidential account information.

Respect user profile visibility and privacy settings.

B. Follow request system

Implement a real follow-request workflow.

When User A opens User B's profile, User A should be able to select:

Follow

This must create a pending follow request.

The request must be delivered to User B.

User B must have two actions:

- Accept Follow Request.
- Reject Follow Request.

When accepted

1. Update the relationship status to accepted.

2. Add User A to User B's followers.

3. Add User B to User A's following list.

4. Update both users' follower/following counts.

5. Enable the appropriate feed visibility.

6. Allow notifications to be delivered according to the user's notification settings.

When rejected

- Mark the request as rejected or otherwise close it safely.
- Do not establish a follower relationship.
- Do not expose private content.
- Do not allow duplicate pending requests.

Relationship states

Use clearly defined states such as:

- Not Following.
- Request Pending.
- Following.
- Request Rejected, where appropriate.

Prevent duplicate follow requests and inconsistent relationships.

Allow users to unfollow accounts they currently follow.

C. Community member discovery

The Surgical Community must allow users to discover other registered and verified professionals.

Provide functional search and browsing using available public profile information.

Users should be able to:

- Browse suggested community members.
- View profile avatars and names.
- Open professional profiles.
- Send follow requests.
- See whether a request is pending or accepted.
- View their followers and following lists.

The community should not be an empty static page.

It must retrieve actual registered users from the backend.

Do not use fake or hard-coded users.

D. Community publication feed

Whenever a user publishes a clinical case successfully, it must appear in the Surgical Community feed.

The feed should retrieve actual published posts from the database.

Each publication should display the appropriate public information, including:

- Author's profile avatar.
- Author's name.
- Specialty or professional title, when available.